

Container Yard: Game Design Document

Game: Container Yard, a third person, over the shoulder arena shooter. **Platform:** Mac (Xogot / Godot 4.7, Mobile renderer). Keyboard and mouse. Fullscreen on a 1920 x 1080 design canvas. **Engine stamp:** `config/features = ["4.7", "Mobile"]` (Xogot 1.7.2 / Godot 4.7.2). Never upgraded by the build. **Art:** Quaternius *Toon Shooter Game Kit* (Dec 2022), CC0, imported at `res://assets/Toon Shooter Game Kit - Dec 2022/` (the pack's own folder name, structure untouched). Every model, with its measured size, is listed in `docs/asset-inventory.md`. **Concept image:** `docs/concept/concept.png` (1600 x 893). It is the acceptance test for the look, see section 1. **Document version:** 1.1, 2026-09-20 (1.0 pre-build specification, sections 0 to 15; 1.1 the full arsenal, section 16). **Status:** Specification. At the time of writing the project contains the asset pack, this document and nothing else: zero scenes, zero scripts.

0. How to read this document

This is the build specification for an implementing agent. Every 3D object named in this document is a real file in the pack; the reference form is the model's base name, for example `Container_Long` means `res://assets/Toon Shooter Game Kit - Dec 2022/Environment/glTF/Container_Long.glTF`, `Character_Soldier` means `res://assets/Toon Shooter Game Kit - Dec 2022/Characters/glTF/Character_Soldier.glTF`, and `AK` means the mesh of that name inside the character files (or `res://assets/Toon Shooter Game Kit - Dec 2022/Guns/glTF/AK.glTF` when a loose weapon model is needed). Only the `glTF/` files are used; the FBX, OBJ and Blend copies stay untouched. Anything the design needs that the pack does not contain is listed in section 14 and the design works without it.

Units, measured from the pack (see `docs/asset-inventory.md`). One Godot unit is one pack unit. A character (`Character_Soldier`, `Character_Enemy`) stands 2.2 units tall on Y 0, shoulders at 1.6, eyes at 1.9; its rest frame is a T pose 2.7 wide. A `Crate` is a 0.79 cube. A `Container_Small` is 2.19 x 2.13 x 2.09, a `Container_Long` is 4.36 x 2.13 x 2.09 (long along X). A `SackTrench` (sandbag wall) is 3.35 wide and 1.28 tall (chest height); `SackTrench_Small` is 2.48 wide and 1.00 tall. `Barrier_Large` and `Barrier_Fixed` are the chain link fence panels with the yellow and black hazard base seen in the concept: 3.81 wide, 3.18 tall, 0.67 deep. `WaterTank_Platform` is only 3.52 tall as shipped and is scaled 2.4 to become the tower. `Tank` is 2.37 x 2.14 x 2.50. `Debris_BrokenCar` is 2.64 x 1.76 x 5.49, long along Z. Trees are 5.1 to 6.2 tall. All of these stand on their origin (min Y about 0).

Coordinate frame. The arena origin is the centre of the yard, X to the right, Y up, Z toward the camera. The player spawns at (0, 0, 6) facing -Z (north). The concept camera looks along -Z. "Further" means more negative Z. Rotations are `rotation_degrees` in Godot's YXZ order, given as (pitch, yaw, roll).

Runtime rules. Every scene is a saved `.tscn` built in the Xogot editor through `xo` with semantic node names (`Player`, `CameraRig`, `Bandit`, `WaveSpawner`, never `Node3D2`). Scripts only add behaviour; they never construct the scene tree. Enemies, projectiles, pickups and effects are instanced from saved prefab scenes. Nothing is created in code that could have been saved as a node.

Terminology. *Bandit*: the single enemy type of 1.0. *Blaster*: the single weapon of 1.0. *Wave*: one batch of bandits; the game has 5. *Breather*: the 5 second pause between waves. *Concept region*: the part of the yard that the concept image shows, authored by hand to match it. *Comparison*: a 1920 x 1080 image with the concept on the left and a game screenshot on the right.

1. Concept match: the look is the acceptance test

`docs/concept/concept.png` is the target. The game is finished visually when a screenshot taken from `ConceptCam` reads as the same picture: same camera, same props at the same places with the same silhouettes, same light, same colours. This section is the most important one in the document and it is checked first in the definition of done (section 15).

1.1 What the concept shows

From the camera's point of view, left to right: a chain link fence panel with a hazard striped base running away from the camera on the left; behind it, stacked crates, a yellow container and a wrecked car in front of a red container; the green soldier in the lower left third, running forward and to the right, firing a red blaster with a yellow flash; two crates in front of the red container; a blue long container stacked on top of the yellow and red ones with sandbags on top and a second red container behind at the upper level; a low sandbag line on the ground at the red container's right end; a tall water tower in the centre back; a yellow tank in the middle distance with two red hooded bandits running toward the player past it; more sandbags on the ground at the right; a second chain link fence panel running toward the camera on the right with a blue container and a yellow container stacked behind it; three crates in the lower right corner; round low poly trees all along the back; orange sand with green grass patches; a pale blue sky turning white at the horizon on the right where the sun is; long hard shadows falling to the left and toward the camera.

1.2 ConceptCam

A Camera3D named `ConceptCam`, saved in `res://scenes/arena.tscn` as a child of the arena root, **perspective** (never orthographic), `fov 50` (Godot vertical FOV, keep aspect height), `near 0.1`, `far 200`, `current false`. Starting pose, to be tuned by the check order in 1.5 and never by eye alone:

- position `(1.4, 2.7, 12.6)`: 1.4 to the right of the player spawn, 2.7 above the ground, 6.6 behind the spawn;
- rotation `(-7, 0, 0)`: looking north along `-Z`, pitched down 7° so the horizon sits at 37 % of the frame height from the top, as in the image.

The gameplay camera at spawn with no input (section 6) must coincide with `ConceptCam` within 0.05 units and 0.5° : the first frame of play *is* the concept frame. `ConceptCam` is the fixed reference; the gameplay rig is tuned to it, not the other way round.

1.3 The `snap_concept_view` action (Tab)

A development only input action `snap_concept_view` bound to Tab. While it is held: the player is teleported to `PlayerSpawn` with the default facing, `ConceptCam` becomes the current camera, the node `ConceptDressing` (1.4) becomes visible, the player plays `Run_Shoot` and the blaster shows its muzzle flash continuously, the HUD hides. On release everything returns to the gameplay state. The action stays in the shipped input map (it costs nothing and the video needs it) and is documented as development only in the README.

1.4 The concept region, authored by hand

The concept region is the part of the yard inside `ConceptCam`'s frustum. It is placed prop by prop at the coordinates in section 7.1, then adjusted against the comparison, never generated, never evenly spaced. `ConceptDressing` is a `Node3D` under the arena root holding what the concept shows but gameplay does not: two static `Character_Energy` instances named `BanditPose1` at `(3.8, 0, -6.5)` and `BanditPose2` at `(6.2, 0, -4.8)`, both facing `(0, 1, 7)` and

playing `Run_Shoot`, each with its `Pistol` mesh visible and the other 13 weapon meshes hidden. `ConceptDressing` is hidden during play and in the menu; it is visible only while `snap_concept_view` is held.

1.5 The comparison protocol

After every visual milestone (build order steps 4, 5, 6, 9, 11, 12 in section 13.2) the agent produces `docs/evidence/comparison-NN.png`, NN counting up from 01 and **never overwriting** an earlier file. Format: a 1920 x 1080 canvas; the concept scaled to 960 x 540 on the left; a game screenshot taken from `ConceptCam` at 1920 x 1080 (`xo editor screenshot --source game --max-resolution 0`, window in the fullscreen 1920 x 1080 canvas) scaled to 960 x 540 on the right; both vertically centred; a 60 px caption strip at the bottom with the file name, the build step and the date. It is produced by `tools/compare.py` (Pillow; the agent writes it in step 1). The final one is saved as `docs/evidence/comparison-final.png`.

Fixed check order. Each comparison is judged in this order and the first failing item is what the next change fixes, nothing else:

1. camera height and horizon line (horizon at 37 % from the top, $\pm 2\%$);
2. distance and FOV (the player's height is 34 % of the frame height, $\pm 3\%$; the water tower's top is at 2 % from the top);
3. subject position (the player's feet at 34 % from the left and 86 % from the top, $\pm 3\%$);
4. main silhouettes (container stack, water tower, tank, wrecked car, sandbag line, both fence panels) at the same screen positions $\pm 4\%$ of the frame width;
5. secondary props (crates, sandbags on the containers, the second red container, trees);
6. sky gradient;
7. ground colour, lit and in shadow;
8. sun direction (shadows fall to the lower left, the right end faces of the containers are lit, the camera facing faces are in shade);
9. fog;
10. glow (muzzle flash and the sun's horizon glow only);
11. grade (saturation, contrast);
12. UI (crosshair, HUD placement does not cover the tower or the tank).

Test for this section: `comparison-final.png` exists, and a person shown the two halves names the same six landmarks in both (container stack, water tower, tank, wrecked car, sandbag line, tree line) and cannot point at a prop that exists in one half only inside the concept region.

1.6 Light matched by value

Values sampled from `concept.png` (sRGB hex, averaged over 12 x 12 pixel patches). The targets are what the **rendered frame** must show; the material albedo is lower than the lit target because the sun adds light. Tune the sun energy and the ambient energy first, then the albedo.

What	Target in the rendered frame	Where it was read
Sky top	#d3e8fa	top edge of the frame
Sky at the horizon	#f6efe3 (near white, warm)	right of the water tower, above the trees
Ground, sunlit	#f8ad6a	open sand, centre right
Ground, in shadow	#8a6a55	the foreground strip and the fence shadow
Ground, deep shadow under a container	#654a3c	below the right container stack

What	Target in the rendered frame	Where it was read
Red container, lit end face	#e8654a	red container, right end
Red container, top and shaded face	#963533	red container, top
Blue container, shaded front face	#2d5284	blue container, front
Blue container, lit end face	#9aa4be	blue container, right end
Yellow container	#a9762c	right stack, upper container
Sandbags	#ac886a	sandbags on the containers
Tree canopy, lit	#b4b542 ; shaded #7f7d2e	right tree
Water tower body	#998689	the tank on top
Tank hull	#c98a2a	tank, upper hull
Soldier uniform	#4f663d (helmet), trousers #e3a339	the player
Bandit hood	#85171e	the nearer bandit

Sun. One `DirectionalLight3D` named `Sun`, `rotation_degrees` `(-32, 125, 0)`: elevation 32°, and the light travels toward `-X`, `+Z` and down, so shadows fall to the left and toward the camera, two and a half to three times the object's height long; the `+X` and `+Y` faces of the containers are lit and the `+Z` (camera facing) faces are in shade, exactly as in the image. Colour `#fff1dc`, energy 1.5, shadows on, `directional_shadow_mode` `PSSM 2 splits`, `directional_shadow_max_distance` 60, `shadow_blur` 0.4 and `light_angular_distance` 0.5 (the concept's shadows are hard edged), `shadow_bias` 0.03.

Environment (`WorldEnvironment` named `Environment`, resource saved at `res://env/yard.tres`): background `Sky` with a `ProceduralSkyMaterial`: `sky_top_color` `#b9d8f4`, `sky_horizon_color` `#f6efe3`, `ground_bottom_color` `#c98a52`, `ground_horizon_color` `#f6efe3`, `sun_angle_max` 12, `sun_curve` 0.15 (the bright patch at the horizon on the right comes from the sun's own glow in the sky material). Ambient light: source `Sky`, `ambient_light_sky_contribution` 1.0, `ambient_light_energy` 0.7. Tonemap `Filmic`, white 6, exposure 1.0. Glow on, intensity 0.3, strength 0.8, bloom 0, HDR threshold 1.0, blend mode `Soft Light` (only the muzzle flash and the sun glow bloom). Fog on, `fog_light_color` `#f0e6d6`, `fog_density` 0.003, `fog_aerial_perspective` 0.3, `fog_sky_affect` 0.2. Adjustments on: brightness 1.0, contrast 1.05, saturation 1.15. SSAO, SSIL, SDFGI, volumetric fog off (Mobile renderer).

Ground. A `MeshInstance3D` named `Ground` with a `PlaneMesh` 160 x 160, `StandardMaterial3D` albedo `#d08a52`, roughness 1.0, saved at `res://materials/ground.tres`, collision box 160 x 0.2 x 160 under it. Grass patches: six `MeshInstance3D` named `Grass1` to `Grass6` with a `PlaneMesh` 3.0 x 2.0 (scaled per instance between 0.6 and 1.4 on `X` and `Z`, rotated between 0° and 180° on `Y`), albedo `#6b7a3b`, at `Y` 0.02, no collision, at `(-2, 0.02, 9)`, `(6, 0.02, 3)`, `(9, 0.02, 8)`, `(-8, 0.02, 7)`, `(4, 0.02, -2)`, `(-3, 0.02, -9)`.

Test: a 12 x 12 patch read from the game screenshot at the same frame position as each row above is within 12 % per channel (sRGB, 0 to 255) of the target for the sky, the sunlit ground and the shaded ground, and within 18 % for the coloured props. `tools/compare.py --values` prints the table with the deltas; the values pass before any grading step is called done.

2. Concept

You are the green soldier holding a container yard against waves of red bandits. The camera sits over your right shoulder; you run between stacked shipping containers, sandbag lines and wrecked cars, aim with the mouse and fire a

blaster that never runs dry. Bandits arrive in waves from the back of the yard, close in on foot and shoot in bursts when they see you; a container or a sandbag wall between you and them blocks their shots, but not yours over the sandbags. Five waves, each bigger than the last, then the results screen and the score to beat. If your health reaches zero the same screen tells you which wave overran you.

Session length: a complete five wave run is 4 to 5 minutes; a run that ends early is shorter. One arena, one difficulty, replayed for a better score.

3. Design pillars

Pillar 1: It looks like the picture

The yard is `docs/concept/concept.png`, built by hand prop for prop, lit by value. Every visual decision is checked against the concept, not against taste. **Test:** the test of section 1.5: `comparison-final.png`, six landmarks named in both halves, no extra or missing prop inside the concept region, and the value table of 1.6 within tolerance.

Pillar 2: The gun feels good

No lag between the mouse and the camera, no lag between the camera and the player's position, a visible shot, a kick, a flash, a puff at the point of impact, a bandit that flinches and a shot sound that is physical, never a beep. **Test (measurable):** (a) camera follow with zero position smoothing: a test script moves the player 1.0 unit in one physics frame and reads the camera's global position in the same frame; the camera moved by the same 1.0 unit (± 0.001). (b) A shot fired at a bandit standing 20 units away in the open registers a hit on the bandit within 2 physics frames of the fire action. (c) The shot sound is not a pure tone: its spectrum has energy above 2 kHz and below 200 Hz in the first 30 ms (checked by `tools/gen_audio.py --verify`, section 10).

Pillar 3: Cover matters

Bandits shoot in straight lines from their gun to the player's belt. Anything solid in between blocks the shot. The sandbag walls are chest height: they block the bandits' shots at the player, and the player, aiming from the camera over the sandbags, still hits the bandits. **Test:** with the player standing 0.8 units behind a `SackTrench` and a bandit 15 units away on the far side at the same height, the player takes zero damage over 10 seconds; the player then steps 2 units sideways into the open and takes damage within 2 seconds. Both halves are one test in `tests/test_cover.gd`.

4. Core gameplay loop

4.1 One wave, second by second

Time	What happens
0.0 s	"WAVE 1" slides in at the top centre of the HUD (0.4 s in, holds 1.2 s, 0.4 s out), the horn sound plays, the combat music loop starts at full volume.
0.5 s	The first bandit appears at a spawn point that is not in the camera's view cone (section 7.4), plays <code>Run_Shoot</code> and paths toward the player. A second bandit follows 2.5 s later (the wave's spawn gap).

Time	What happens
3.0 s	The bandit reaches its preferred range of 9 units with line of sight, stops, plays <code>Idle_Shoot</code> and fires a burst of 3 shots 0.15 s apart, then waits 2.0 s.
3.4 s	The player, moving sideways behind the sandbag line, aims the crosshair on the bandit and holds fire: 8 shots per second, 10 damage each, tracer, muzzle flash, recoil kick, impact puff on the bandit, <code>HitReact</code> on the bandit. Third hit: the bandit plays <code>Death</code> , its score of 100 pops at the top left, the kill sound plays.
5.0 s	The next bandit is already closing in from the tank's side. The player moves to keep a container corner between them, peeks and shoots. Health lost to the burst at 3.0 s shows as a red flash on the frame edge and the health bar at 82.
24 s	The last bandit of wave 1 falls. "WAVE CLEARED, +250" pops, the breather starts: 5 seconds, health regenerates to 100 at 20 points per second, the music drops to 60 % volume, two <code>Health</code> pickups appear at their spawn spots from wave 2 on.
29 s	"WAVE 2" slides in. Six bandits this time, three alive at once.

4.2 What makes it repeatable

- **Score.** 100 per kill, a wave bonus of 250 times the wave number, and at the end 5 points per health point left. The best score and the best wave reached are saved and shown on the menu.
- **Waves are fixed.** The counts, gaps and spawn order per wave come from the table in section 12 with a fixed seed (`4000 + wave`), so a replay is the same problem and improvement is measurable.
- **Cover is positional.** Where the player stands decides how much damage comes in. The yard has good spots (behind the sandbag line, the container corner) and bad ones (the open middle), and the waves push the player from one to the next.

5. Controls

Keyboard and mouse only in 1.0. The mouse is captured (`MOUSE_MODE_CAPTURED`) during play and released in every menu.

Action name	Binding	Effect
<code>move_forward</code> / <code>move_back</code> / <code>move_left</code> / <code>move_right</code>	W / S / A / D, plus the arrow keys	Move relative to the camera's yaw (section 7.2).
<code>fire</code>	Left mouse button	Hold to fire the blaster at 8 shots per second.
<code>aim</code>	Right mouse button	Hold to aim: camera FOV narrows from 50 to 40 over 0.12 s, the crosshair shrinks, the player's facing locks to the camera yaw (section 7.2).
<code>pause</code>	Escape	Pause overlay, mouse released.
<code>snap_concept_view</code>	Tab	Development only, section 1.3.
Mouse motion		Camera yaw and pitch, 0.1° per pixel, no acceleration, no smoothing. Pitch clamped between <code>-40°</code> (looking down) and <code>+20°</code> (looking up).

All actions are defined in the project's input map (through `xo inputmap`) before any script reads them. Menu buttons are standard `Button` nodes with the press squash animation (scale 0.94 for 0.08 s) and the click sound.

6. Camera

- **Type:** one third person over the shoulder rig per player, `Camera3D` named `Camera` inside `CameraRig` inside `Player` (section 8). Because the rig is a child of the player, the follow position is exact by construction: **there is no position smoothing, no lerp, no spring on the follow position** (input lag is the enemy of pillar 2). The only smoothing allowed is on rotation, and 1.0 uses none: yaw and pitch apply the mouse delta in the same frame.
 - **Rig geometry:** `CameraRig` (`Node3D`) at `(0, 1.6, 0)` in the player (pivot at shoulder height); it carries the yaw. `Pitch` (`Node3D`) inside it carries the pitch, default -7° . `Arm` (`SpringArm3D`) inside `Pitch`, `spring_length` 6.6, `margin` 0.3, collision mask layer 1 (world), so the camera pulls in when a container is behind the player; `Camera` inside `Arm` at local `(1.4, 1.1, 0)`: 1.4 to the right (over the right shoulder, the player sits in the left third of the frame as in the concept), 1.1 up. FOV 50, near 0.1, far 200. With the player at spawn, no input, this places `Camera` at `(1.4, 2.7, 12.6)` pitched -7° , which is `ConceptCam` (1.2). Aiming (RMB) narrows the FOV to 40 and shortens the arm to 5.2 over 0.12 s; releasing restores both over 0.12 s.
 - **Test:** `tests/test_camera.gd` teleports the player by `(1, 0, 0)` and asserts the camera's global position moved by exactly `(1, 0, 0)` in the same physics frame; then reads the camera's global transform at spawn with no input and asserts it equals `ConceptCam`'s within 0.05 units and 0.5° .
 - **Shake:** 0.15 s, 0.06 units, on the player taking a hit; 0.3 s, 0.12 units on the player's death. Shake is applied as an offset on `Camera.h_offset / v_offset`, never on the rig's position.
 - **Menu camera:** `MenuCam` (`Camera3D`, FOV 45) in `res://scenes/ui/main_menu.tscn`, which instances the arena. It orbits the yard: radius 24, height 7.5, looking at `(0, 1.5, -4)`, one revolution every 48 s, starting at the concept's side (azimuth from +Z), continuous, no cut. The menu is never a static image and never a frozen level: the trees and the flags of the orbit move, the sun casts shadows, the two `ConceptDressing` bandits are hidden. **Test:** two menu screenshots 3 s apart differ (mean absolute pixel difference above 2 on a 0 to 255 scale).
-

7. Systems

7.1 Arena layout

`res://scenes/arena.tscn`, root `Arena` (`Node3D`). The playable yard is 38 x 38 units (`x` and `z` from -19 to 19), fenced all round; outside the fence lies dressing out to the 160 unit ground plane. Everything inside is placed by hand. Coordinates are the model origin (on the ground unless a `Y` is given); rotation is the `Y` rotation in degrees unless a full triple is given. Every reachable prop has collision (a `StaticBody3D` with a `CollisionShape3D` box fitted to the model's AABB, or `create_trimesh_collision` for the car and the tank). Props sit in **clusters**, never evenly spaced singles.

Perimeter. `Barrier_Large` panels, 3.81 wide, ten per side, centres from -17.1 to 17.1 in steps of 3.8, at `Z = -19` (rotation 0), `Z = 19` (rotation 180), `X = -19` (rotation 90), `X = 19` (rotation -90); the four corners get a `Barrier_Fixed` at 45° . Two panels are replaced by `Barrier_Single` low barriers to make the two bandit gates: at `(-9.5, 0, -19)` and `(19, 0, -5.7)`. Behind the perimeter (outside), dressing with no collision needed: `Tree_1` to `Tree_4` in a loose ring at 22 to 30 units from the origin, 14 trees in total, never two of the same model next to each other, and four `StreetLight` at `(-21, 0, -21)`, `(21, 0, -21)`, `(21, 0, 21)`, `(-21, 0, 21)`.

Concept region (inside `ConceptCam`'s frustum; these positions produce the concept and are tuned only through the comparison protocol):

Node name	Model	Position	Rotation / scale	Notes
FenceLeftA	Barrier_Large	(-6.4, 0, 3.6)	90	runs along Z, its post is the nearest thing on the left of the frame
FenceLeftB	Barrier_Large	(-8.3, 0, 1.7)	0	runs along X behind the first, seen above the crates
CrateStackLeft	3 x Crate	(-8.0, 0, 2.6), (-8.8, 0, 2.9), (-8.4, 0.79, 2.7)	0, 15, 8	behind the left fence
ContainerYellowLeft	Container_Small, yellow variant	(-4.6, 0, -5.5)	0	behind the car
WreckedCar	Debris_BrokenCar	(-4.0, 0, -1.6)	75	side on, nose toward the fence
CratesFront	3 x Crate	(-1.8, 0, -2.6), (-1.0, 0, -2.9), (-1.5, 0.79, -2.7)	0, 12, 5	in front of the red container
ContainerRedGround	Container_Long (red as shipped)	(0.8, 0, -5.5)	0	the centre of the frame
ContainerBlueUpper	Container_Long, blue variant	(-2.4, 2.13, -5.5)	0	on top of the yellow and the left half of the red
SandbagsUpperA	SackTrench_Small	(1.8, 2.13, -5.2)	0	on the red container's roof, right of the blue
SandbagsUpperB	SackTrench	(2.8, 2.13, -5.9)	10	behind the first
SandbagsUpperLeft	SackTrench_Small	(-5.2, 4.26, -5.5)	0	on top of the blue container's left end
ContainerRedBack	Container_Small (recoloured to the red #9b3e3c variant)	(2.6, 2.13, -8.4)	0	the second red container seen behind at the upper level
SandbagLine	SackTrench	(4.8, 0, -3.2)	20	the chest height cover at the red container's right end
SandbagLineB	SackTrench_Small	(7.6, 0, -2.4)	35	continues the line
SandbagsRight	SackTrench_Small	(9.0, 0, -6.0)	-20	by the tank
WaterTower	WaterTank_Platform	(3.5, 0, -14.0)	scale (2.4, 2.4, 2.4)	8.4 tall; its top must reach 2 % from the top of the frame
TankYellow	Tank	(5.5, 0, -8.5)	-30	barrel pointing to the front left, as in the image
FenceRightA	Barrier_Large	(7.2, 0, 3.4)	-90	runs along Z, its post at 81 % of the frame width
FenceRightB	Barrier_Large	(7.2, 0, -0.4)	-90	continues it
ContainerBlueRight	Container_Long, blue variant	(10.6, 0, 0.8)	90	behind the right fence, long along Z
ContainerYellowRight	Container_Long, yellow variant	(10.6, 2.13, 1.4)	90	on top of it, slightly toward the camera

Node name	Model	Position	Rotation / scale	Notes
CratesRight	3 x Crate	(7.6, 0, 6.2), (8.5, 0, 6.6), (8.0, 0.79, 6.4)	0, 20, 10	lower right corner of the frame
TiresRight	Debris_Tires	(6.6, 0, 1.5)	0	stands in for the rocks (section 14)
TreesBack	Tree_1, Tree_2, Tree_3, Tree_4, Tree_1, Tree_2	(-13, 0, -12), (-8, 0, -14), (1, 0, -16), (7, 0, -17), (11.5, 0, -13), (14, 0, -8)	0, 40, 90, 20, 130, 60	the tree line; the two nearest the tower must show above the containers
TreesLeft	Tree_3, Tree_1	(-12, 0, -4), (-14, 0, 2)	70, 0	the two big trees behind the left fence

Container colour variants. The pack ships `Container_Long` in red (Red `#9b3e3c`) and `Container_Small` in blue (Blue `#42549b`). The concept needs blue and yellow long containers and a yellow small one. The build saves three `StandardMaterial3D` at `res://materials/container_red.tres` (`#9b3e3c`), `container_blue.tres` (`#42549b`) and `container_yellow.tres` (`#c9952a`), all roughness 0.9, and assigns them as `surface_material_override` on surface 0 of the container instance (surface 1, Grey, stays). Same mechanism for any prop that needs a colour change; the pack files are never edited.

The rest of the yard (outside the concept frustum, still by hand, still clustered): a second sandbag cluster `SackTrench` x2 at `(-8, 0, 10)` and `(-11, 0, 8)`; a container corner at the north west, `Container_Long` red at `(-12, 0, -13)` rotation 30 with `Container_Small` blue at `(-9, 0, -11)`; a crate and barrel cluster at the north east, `Crate` x4 with `ExplodingBarrel` x2 (decoration only, they do not explode in 1.0) at `(12, 0, -14)`; `Barrier_Trash` at `(14, 0, 12)`; `CardboardBoxes_3` and `CardboardBoxes_2` at `(-14, 0, 14)`; `Pallet` x2 and `WoodPlanks` at `(2, 0, 14)`; `TrafficCone` x3 near the south gate at `(16, 0, 16)`. Nothing is placed on a grid.

Navigation. `NavigationRegion3D` named Nav under Arena, `NavigationMesh` with `agent_radius` 0.5, `agent_height` 2.2, `agent_max_climb` 0.3, `cell_size` 0.25, parsed from the static colliders of the Arena subtree (geometry source: static colliders, layer 1), baked in the editor and saved with the scene. The fence colliders are part of the bake, so the mesh ends at the fence.

Test, differential: `tests/test_arena.gd` loads the arena, checks that every child of `Props` and `Perimeter` has a `StaticBody3D` with at least one `CollisionShape3D`, that the baked navigation mesh exists and has more than 200 polygons, and that no navigation polygon vertex lies outside `|X| < 18.6`, `|Z| < 18.6`. The enemy bounds test is in 7.4.

Test, screenshot: an editor screenshot from `ConceptCam` (xo editor screenshot `--source viewport --view-target ...` is not used; the camera is activated and `--source game` at 1920 x 1080 is), producing `comparison-01.png` as soon as the concept region stands, before lighting.

7.2 Player controller

`res://scenes/actors/player.tscn`, root `Player` (`CharacterBody3D`), collision layer 2, mask 1 (world). `CollisionShape3D` named `Shape`, capsule radius 0.45, height 2.0, centred at Y 1.0. `Model` (`Node3D`) holds the `Character_Soldier` instance, scale 1.0, with all weapon meshes hidden except AK, which is the blaster (7.3). `AimPoint` (`Marker3D`) at `(0, 1.0, 0)`: the point enemies shoot at. `CameraRig` as in section 6. `Muzzle` (`Marker3D`) is a child of the AK mesh's bone attachment, at the barrel's end (local `(0.55, 0.3, 0)` on the mesh, adjusted once against a screenshot so the flash sits at the barrel).

Movement. Speed 6.5 units per second in every direction, ramp up over 0.08 s and down over 0.06 s (no sliding). Directions are relative to the camera yaw: W moves along the camera's flattened forward, D along its right. Gravity 24 units per second squared, `move_and_slide`, the player never leaves the ground in 1.0 (no jump). Animation: `Idle` when still, `Run` when moving and not aiming or firing, `Run_Shoot` when moving while aiming or firing, `Idle_Shoot` when still and firing, `HitReact` overlay on damage is not used on the player (the camera shake and the frame flash carry it), `Death` on death. Blend time 0.1 s between clips.

Facing. While `aim` or `fire` is held the player's yaw equals the camera yaw, instantly (the body turns with the mouse, strafing sideways and backwards). When neither is held the player faces its movement direction, turning at 720° per second, and keeps the last facing when still. The blaster fires along the camera's aim, never along the body, so a shot always lands at the crosshair.

Three facing tests, `tests/test_facing.gd`, each starting from a fresh arena with the player at spawn (`(0, 0, 6)`, facing `-Z`, camera yaw 0):

1. *From spawn.* Press `fire` for one frame. Assert the player's forward (`-global_basis.z`) is `(0, 0, -1)` within 2°, and that the hitscan from the camera through the crosshair centre hits the point `(0, 1.4, -14)` when a 1 x 1 test wall is placed there.
2. *Camera rotated 90° to the right* (inject mouse motion of 900 px to the right, the camera now looks along `+X`). Hold `fire` and press `move_forward` for 30 physics frames. Assert the player's forward is `(1, 0, 0)` within 2° and the player's X increased by at least 2.5 units while Z changed by less than 0.3.
3. *Camera rotated 180°* (1800 px, looking along `+Z`). Hold `fire`, press `move_right` for 30 frames. Assert the forward is `(0, 0, 1)` within 2° and the player moved along `-X` by at least 2.5 units (camera relative strafing: right of a camera that looks along `+Z` is `-X`).

Health. 100 points. Bandit hits take 6. Below 30 the health bar pulses red and a heartbeat sound plays every 1.2 s. At 0: `Death` clip, camera shake 0.3 s, music stops, 1.5 s later the results screen with "Overrun at wave N". Regeneration only during the breather: 20 points per second up to 100. `Health` pickups (7.5) give 35.

Damage feedback. A red vignette (a full screen `ColorRect` with a radial shader, alpha 0.35, fading over 0.4 s), the hit sound, camera shake 0.15 s, and a directional marker: a 40 px arc at the screen edge toward the shooter for 0.6 s.

7.3 The blaster (the single weapon)

The weapon is a prefab so a second one can be added later: `res://scenes/weapons/blaster.tscn`, root `Blaster` (`Node3D`), instanced under `Player/WeaponSlot` (`Node3D`, the only weapon slot; a second weapon prefab and a swap key are reserved for the on camera iteration, section 13.1, nothing more than the slot and an exported `weapon_scene` on the player is prepared for it). The visible gun is the `AK` mesh already inside the character (bone `Index1_R`), with its `Wood` surface (surface 2) overridden by `res://materials/blaster_red.tres` (`#c0392b`) so it reads as the concept's red blaster.

Property	Value
Type	Hitscan (a <code>PhysicsDirectSpaceState3D</code> ray from the camera through the crosshair, max 60 units, mask world + enemies).
Rate of fire	8 shots per second while <code>fire</code> is held (0.125 s between shots), first shot immediate.
Damage	10 per hit; a bandit has 30 (three hits). Head bone hits do not matter in 1.0.
Spread	a cone of 1.2° (hip), 0.5° while aiming; the deviation is uniform inside the cone.
Ammo	infinite, no reload, no heat.

Property	Value
Recoil	the camera pitch kicks up 0.6° on each shot and returns over 0.12 s (spring, no overshoot); the yaw drifts randomly $\pm 0.2^\circ$. The AK mesh kicks back 0.06 units along its barrel for 0.06 s.
Tracer	a MeshInstance3D line (a BoxMesh 0.03 x 0.03, scaled to the hit distance, unshaded, #ffd36b, additive) from Muzzle to the hit point, visible for 0.05 s. Instanced from res://scenes/fx/tracer.tscn.
Muzzle flash	res://scenes/fx/muzzle_flash.tscn: a billboard QuadMesh 0.5 x 0.5 with a code drawn star texture (section 14), colour #ffe066, unshaded, and an OmniLight3D 3 units range, energy 4, #ffb347, both for 0.05 s. The concept's yellow flash.
Impact	res://scenes/fx/impact.tscn: a one shot GPUParticles3D, 8 particles, SphereMesh 0.06, #d9b48a on the ground and props, #ff5a4a on a bandit, 0.3 s lifetime, spread along the hit normal.
Sound	the blaster shot (section 10), pitch varied $\pm 6\%$ per shot.

Test: tests/test_weapon.gd places a bandit 20 units in front of the camera, injects fire for 1.0 s and asserts the bandit received 8 ± 1 hits (spread never misses a 2.2 tall target at 20 units with a 1.2° cone, because the cone's radius there is 0.42 units) and died within the first 0.4 s; then asserts the tracer node existed for at most 4 frames.

7.4 The bandit (the single enemy type, the single behaviour)

res://scenes/actors/bandit.tscn, root Bandit (CharacterBody3D), collision layer 3, mask 1 (world) + 3 (other bandits, so they do not overlap). Capsule radius 0.45, height 2.0 at Y 1.0. Model holds Character_Enemy with the Pistol mesh visible and the other 13 weapon meshes hidden. NavAgent (NavigationAgent3D, radius 0.5, path_desired_distance 0.6, target_desired_distance 1.0, avoidance on with 8 max neighbours). Muzzle (Marker3D) at (0.3, 1.3, 0.5) in the model. Eyes (RayCast3D) from (0, 1.5, 0) toward the player's AimPoint, mask world only, used for line of sight. HurtBox is the body itself (the blaster's ray hits the capsule).

The one behaviour: advance and shoot.

State	Animation	Rule
Spawn	Run_Shoot	Appears at a spawn marker with a 0.3 s scale pop (0 to 1, ease out). Chooses the player as its navigation target.
Advance	Run_Shoot	Moves along the navigation path at 4.0 units per second, replanning every 0.5 s, facing its velocity. Enters Hold when the straight line distance to the player is at most 9 units and Eyes sees the player's AimPoint (no world collider in between).
Hold	Idle_Shoot	Stands, turns to face the player at 360° per second. Fires a burst of 3 shots 0.15 s apart, then waits 2.0 s, then bursts again, as long as it sees the player. Returns to Advance when the distance exceeds 11 units or the line of sight is lost for more than 0.4 s.
Hit	HitReact (0.43 s, overlay on the upper body if the build uses a blend tree, otherwise a full clip)	On each blaster hit. Movement continues.
Death	Death (0.77 s)	Health reached 0. The collider is disabled, the score pops, the kill sound plays, the body sinks 0.8 units over 0.8 s starting 1.0 s after death, then the node is freed.

Bandit shots. Each shot is a hitscan ray from Muzzle toward the player's AimPoint with a uniform random deviation inside a 5° cone; at 9 units the cone radius is 0.79 units, so roughly half the shots hit the 0.45 radius capsule when the player is in the open. A shot whose ray hits a world collider before the player is blocked: impact puff on the cover, no damage. Damage 6 per hit. Tracer and muzzle flash as the blaster's but #ff8a5a, sound: the bandit's pistol shot (section 10), pitch varied $\pm 8\%$.

Spawning. WaveSpawner (Node under Arena, script) owns six Marker3D under Arena/SpawnPoints: SpawnN1 (-9.5, 0, -17.5), SpawnN2 (6, 0, -17.5), SpawnE1 (17.5, 0, -5.7), SpawnE2 (17.5, 0, 8), SpawnW1 (-17.5, 0, -8), SpawnS1

(-14, 0, 17). Each spawn picks, with the wave's seeded random, one of the points that are **outside the camera's view cone** (the dot product between the camera's flattened forward and the direction to the point is below 0.3, or the point is farther than 26 units), and at least 12 units from the player; if none qualifies the farthest point is used. Bandits never spawn in view.

Test, differential: `tests/test_bandit_bounds.gd` runs the arena for 60 simulated seconds with the player standing still at spawn (invulnerable for the test) and all five waves forced to spawn at their normal gaps. Every physics frame it asserts for every living bandit: $|X| < 18.6$, $|Z| < 18.6$, $-0.1 < Y < 0.6$, and a sphere overlap query (radius 0.35 at the bandit's chest, Y 1.2, mask world) returns zero hits (the bandit is never inside a container, the tank, the car or the fence). **The test also requires each bandit to have accumulated at least 8 units of travelled path within its first 10 seconds alive; a bandit that never moves fails the test.** Without that clause a spawner that parks every bandit at its spawn point would pass, so the clause is what makes the test meaningful.

Test, line of sight: the pillar 3 test (`tests/test_cover.gd`).

7.5 Pickups

`res://scenes/actors/health_pickup.tscn`, root `HealthPickup` (`Area3D`, layer 4, monitoring the player's layer 2), the `Health` model rotating at 90° per second and bobbing ± 0.1 units over 1.6 s, an `OmnLight3D` #8dff8d energy 1.2 range 2. Touching it gives 35 health (capped at 100), plays the pickup sound and a green particle burst, and frees the node. Two pickups appear at the start of every breather from wave 2 on, at `PickupA` (-6, 0, 9) and `PickupB` (9, 0, -10) (`Marker3D` under `Arena/PickupPoints`), only if that spot is empty. The `Health` model is the pack's medkit.

7.6 Scoring

- **Kill:** 100. Shown as a "+100" pop at the score for 0.6 s.
- **Wave bonus:** 250 x the wave number, awarded when the last bandit of the wave dies.
- **Survival bonus:** 5 x the health left after wave 5.
- **Best score** and **best wave** are saved (7.8) and shown on the menu.

7.7 Fail and win states

- **Overrun:** the player's health reaches 0. `Death` clip, 1.5 s, results screen titled "Overrun at wave N" with the score.
- **Yard held:** the last bandit of wave 5 dies. 2.0 s of slow motion (`Engine.time_scale` 0.4) with the camera holding, then the results screen titled "Yard held" with the score, the survival bonus and the time.
- Either way the run can be restarted from the results screen without passing through the menu.

7.8 Persistence

`user://save.json`: {best_score, best_wave, runs, music, sfx}. Written on every results screen and on every settings change. Read on start. A missing or corrupt file means zeros and both audio toggles on.

8. Scene and node architecture

All scenes are created in the Xogot editor through `xo`. Node names are the ones listed here.

Scene	Root	Owns
res://scenes/main.tscn	Main (Node)	SceneSlot (Node) holding the current screen; Fader (CanvasLayer, layer 100) with Rect (ColorRect, black, fade 0.3 s). Script main.gd: switch_to(path). Main scene of the project.
res://scenes/arena.tscn	Arena (Node3D), script arena.gd (run controller)	Environment (WorldEnvironment), Sun (DirectionalLight3D), Ground, Grass (Node3D with Grass1 to Grass6), Perimeter (Node3D: the fence panels), Props (Node3D: every container, sandbag, crate, car, tank, tower, tree, light, each an instanced model with its StaticBody3D), ConceptCam (Camera3D), ConceptDressing (Node3D: BanditPose1, BanditPose2), Nav (NavigationRegion3D), PlayerSpawn (Marker3D at (0, 0, 6)), SpawnPoints (Node3D with the six markers), PickupPoints (Node3D with two markers), Player (Player instance), Bandits (Node3D, runtime parent for bandit instances), Pickups (Node3D), Effects (Node3D, runtime parent for tracers, flashes, impacts), WaveSpawner (Node, script wave_spawner.gd), HUD (HUD instance).
res://scenes/actors/player.tscn	Player (CharacterBody3D), script player.gd	Shape, Model (Character_Soldier), AimPoint, CameraRig / Pitch / Arm / Camera, WeaponSlot (Node3D) holding a Blaster instance, Muzzle (under the AK bone attachment), Sounds (Node3D with the AudioStreamPlayer3D nodes: Shot, Hit, Step, Heartbeat).
res://scenes/weapons/blaster.tscn	Blaster (Node3D), script blaster.gd	Exported numbers of 7.3 (rate, damage, spread, recoil); FlashPoint (Marker3D). The mesh lives in the character, the prefab only owns behaviour and numbers, so a second weapon prefab can replace it.
res://scenes/actors/bandit.tscn	Bandit (CharacterBody3D), script bandit.gd	Shape, Model (Character_Enemy), NavAgent, Eyes (RayCast3D), Muzzle, Sounds (Shot, Hurt, Die).
res://scenes/actors/health_pickup.tscn	HealthPickup (Area3D), script health_pickup.gd	Shape, Model (Health), Light.
res://scenes/fx/tracer.tscn, muzzle_flash.tscn, impact.tscn	Tracer (MeshInstance3D), MuzzleFlash (Node3D), Impact (GPUParticles3D)	One shot effects, freed by their own timers.
res://scenes/ui/main_menu.tscn	MainMenu (Node3D), script main_menu.gd	An Arena instance with Player, HUD and WaveSpawner disabled and ConceptDressing hidden; MenuCam (Camera3D, orbiting, section 6); UI (CanvasLayer: Title, Subtitle, Play, Best, Music, Sound, Quit, Credit).
res://scenes/ui/hud.tscn	HUD (CanvasLayer), script hud.gd	Crosshair (Control, code drawn), Health (ProgressBar 360 x 22 with a Label), Wave (Label), Score (Label), Enemies (Label, "3 left"), Message (Label, centre, the wave announcements), DamageVignette (ColorRect, shader), HitMarker (Control, the screen edge arc), PauseMenu

Scene	Root	Owns
		(Control: Dim, Panel with Resume, Restart, Menu, Music, Sound).
res://scenes/ui/results.tscn	Results (Control), script results.gd	Background (the last frame, dimmed), Panel: Title, Score, Stats (waves, kills, time, best), Buttons (Retry, Menu).

Autoloads: Game (res://scripts/autoload/game.gd): save data, scene switching, run results hand off, the wave table.
Audio (res://scripts/autoload/audio.gd): SFX pool with pitch variation and the music player with crossfade.

Signals (up) and calls (down).

- `Blaster.fired(origin, hit_point, hit_body)` → Player → `Arena.on_player_shot` (spawns the tracer, the flash, the impact, applies damage to a `Bandit`).
- `Bandit.died(bandit)` → `WaveSpawner.on_bandit_died` → `Arena` updates the score and the HUD.
- `Bandit.fired(origin, hit_point, hit_player: bool)` → `Arena.on_bandit_shot`.
- `Player.damaged(amount, from_position)`, `Player.died` → `Arena` (vignette, marker, shake, overrun).
- `WaveSpawner.wave_started(n)`, `wave_cleared(n)`, `all_waves_cleared` → `Arena` (announcements, bonus, breather, pickups, win).
- `Arena.run_finished(result: Dictionary)` → `Game.run_complete(result)` → results screen.

Collision layers: 1 world (ground, props, fence), 2 player, 3 bandits, 4 pickups. Rays from the blaster use mask 1 + 3; rays from the bandits use mask 1 + 2; Eyes uses mask 1.

9. Art direction

Look. The concept: flat coloured low poly, warm afternoon sun, hard long shadows, saturated primaries on the containers against orange sand and a pale sky. All of it is specified by value in section 1.6 and checked by the comparison protocol; this section lists what goes where.

Models by role (all from the pack's glTF/ folders):

- Characters: `Character_Soldier` (player), `Character_Enemy` (bandit). `Character_Hazmat` unused in 1.0.
- Weapons: the AK mesh inside `Character_Soldier` (blaster, red override), the `Pistol` mesh inside `Character_Enemy`. Loose gun files unused.
- Structure and cover: `Container_Long`, `Container_Small` (with the three colour materials), `SackTrench`, `SackTrench_Small`, `Barrier_Large`, `Barrier_Fixed`, `Barrier_Single`, `Barrier_Trash`, `WaterTank_Platform` (scaled 2.4).
- Vehicles: `Tank`, `Debris_BrokenCar`.
- Props: `Crate`, `CardboardBoxes_2`, `CardboardBoxes_3`, `Pallet`, `WoodPlanks`, `ExplodingBarrel` (decoration), `TrafficCone`, `Debris_Tires`, `StreetLight`, `Health` (pickup).
- Trees: `Tree_1`, `Tree_2`, `Tree_3`, `Tree_4`.

Weapon meshes inside the characters. Every character file carries 14 weapon meshes on the right hand, all visible when instanced. The player scene hides all but AK; the bandit scene hides all but Pistol; `ConceptDressing` does the same. A test (tests/test_scenes.gd) asserts exactly one weapon mesh is visible in each character instance.

Feedback effects (no pack assets needed): tracer, muzzle flash, impact puff, hit react, death sink, score pops, wave announcement slide, damage vignette and edge marker, camera shake, health pickup burst, results slow motion, screen fade between scenes.

Unused files. The pack's `FBX/`, `OBJ/` and `Blends/` duplicates, the loose `Guns/gLTF` models, `Character_Hazmat`, the sofas, the traps, the landmine, the key, the sign, the structures (`Structure_1` to `Structure_4` are prebuilt piles that would fight the hand authored concept region) and the papers stay in the project untouched.

10. Audio plan

The pack contains no audio. Every sound is a 16 bit 44.1 kHz WAV synthesised by `tools/gen_audio.py` (Python, standard library only, deterministic seed) and committed under `res://assets/audio/sfx/` and `res://assets/audio/music/`. The script also has a `--verify` mode that checks the three rules below on its own output and exits non zero if any fails.

Three hard rules.

- No chiptune, no 8 bit.** No square or pulse waves as a lead, no bit crushing, no arpeggiated chip leads. Instruments are built from layered sines, saws and filtered noise with envelopes and a touch of reverb (a short convolution or a feedback delay), so they read as physical and warm.
- Every music loop is seamless and has at least 45 seconds of distinct material** before it repeats. Seamless means the last 20 ms crossfade into the first 20 ms with no click and no gap, and the harmonic content at the loop point continues (the loop point falls on a bar line). Distinct means no bar is a copy of an earlier bar in the same loop.
- SFX are physical and satisfying, with random pitch variation.** Every SFX has a transient (an attack under 5 ms), a body and a tail, and is played through `Audio` with a random pitch scale between 0.94 and 1.06 ($\pm 8\%$ for the bandit's pistol) so no two shots sound identical.

Event	Sound	Notes
Blaster shot	a 90 ms blast: a noise burst with a 4 kHz to 400 Hz sweep, a 120 Hz sine thump, a short metallic tail	pitch $\pm 6\%$ per shot, the core sound of the game
Bandit pistol shot	a drier, shorter crack, 60 ms	pitch $\pm 8\%$
Bullet impact on props	a short knock with a filtered noise tail, 3 variants (metal, wood, sand) chosen by the hit body's group	
Bullet hit on a bandit	a soft thud plus a short high snap	
Player hit	a low thump with a 200 ms muffled tail, plus a 0.3 s low pass on the music	
Heartbeat (health below 30)	a two beat low thump every 1.2 s	
Bandit death	a falling two note grunt like tone and a body thud	not a voice sample, synthesised
Health pickup	a bright rising chime, 0.4 s	
Wave start	a horn: a 1.2 s brass like swell built from detuned saws through a low pass sweep	
Wave cleared	a short three note rising fanfare, 1.0 s	
Yard held	a 5 s sting, major, with the horn	
Overrun	a 3 s descending sting	

Event	Sound	Notes
Footsteps	a soft sand crunch every 0.32 s while running, 4 variants	
UI click and hover	a wooden tick	
Music: menu	60 s loop, 88 BPM, a warm desert afternoon: plucked bass, slow pad, brushed percussion, a distant guitar like lead	rule 2 applies
Music: combat	48 s loop, 118 BPM, driving toms and kick, a saw bass line, stabs on the off beats	drops to 60 % volume during breathers over 0.5 s
Music: results	the 5 s sting, then the menu loop at 50 %	

Buses: Master, Music, SFX. The pause menu and the main menu toggle mute per bus. The music ducks by 6 dB while the wave cleared fanfare plays.

Test: `python3 tools/gen_audio.py --verify` passes: every music file loops seamlessly (peak sample difference across the loop point below 2 % of full scale), is at least 45 s long with no repeated bar, contains no square wave lead (no file has more than 30 % of its energy in odd harmonics of a single fundamental for more than 2 s), and every SFX has an attack under 5 ms and energy both above 2 kHz and below 200 Hz in its first 30 ms.

11. UI and HUD

Launch settings, written before any UI is laid out (already in `project.godot` at the time of writing): design canvas 1920 x 1080 (`display/window/size/viewport_width 1920`, `viewport_height 1080`), `display/window/stretch/mode canvas_items`, `display/window/stretch/aspect expand`, `display/window/size/mode 3` (fullscreen on Mac), MSAA 3D 2x, Mobile renderer. Every `Control` is anchored, never positioned by absolute pixels from the top left of a fixed window, so a 16:10 Mac screen shows more width, never a cropped or stretched HUD. Font: Godot's default font with a 3 px dark outline until a font is added (section 14). Body 32 px, HUD numbers 40 px, titles 96 to 140 px.

11.1 Main menu

The live orbiting yard behind (section 6). Title "CONTAINER YARD" (140 px, top centre, slight idle bob ± 3 px over 2 s), subtitle "hold the yard · five waves · one blaster", `Play` (400 x 110 px, centre), best line "Best: 3 250 · wave 5", `Music` and `Sound` toggles, `Quit`, credit "Models by Quaternius (CC0) · quaternius.com" bottom right. The mouse is visible here.

11.2 HUD (in a run)

- Top left: `Score` with the count up and the "+100" pops.
- Top centre: `Wave` ("Wave 2 / 5") and `Enemies` ("4 left"); `Message` under them for the announcements.
- Bottom left: `Health` bar 360 x 22 px, green above 60, orange to 30, red and pulsing below, with the number.
- Centre: `Crosshair`, code drawn: four 14 px lines with a 6 px gap that widens to 12 px while moving and closes to 3 px while aiming, plus a 2 px dot; turns red for 0.1 s on a hit.
- Frame edges: `DamageVignette` and `HitMarker`.
- Nothing covers the water tower or the tank in the concept frame (check item 12 of section 1.5).

11.3 Pause

Dim overlay (black, alpha 0.55), panel with `Resume`, `Restart`, `Menu`, `Music`, `Sound`. The mouse is released; the game tree is paused (`process_mode` of the HUD set to `Always`).

11.4 Results

Title "Yard held" or "Overrun at wave N", the score counting up over 1.2 s, stats (waves cleared, kills, time, best score with "new best!" when beaten), `Retry` and `Menu`. Behind it the last frame, dimmed.

12. Content plan: the five waves

Wave	Bandits	Max alive	Spawn gap (s)	Bandit speed (u/s)	Preferred range (u)	Burst pause (s)
1	4	2	2.5	4.0	9	2.0
2	6	3	2.0	4.0	9	2.0
3	8	4	1.8	4.2	8	1.8
4	10	4	1.5	4.4	8	1.6
5	12	5	1.2	4.6	7	1.4

Rules: a wave ends when its last bandit dies; the breather is 5 s; spawn points are chosen with the seed `4000 + wave` under the rule of 7.4; a bandit is never spawned while `Max alive` are alive. Expected wave durations for a competent player: 25, 35, 45, 55, 65 s, plus four breathers: about 4 min 45 s in total. Tuning after the first full playthrough changes counts, never the rules.

Tutorial: wave 1 shows three timed messages in `Message`: "WASD to move, mouse to aim, hold left click to fire" (at 0 s, 4 s), "Sandbags block their shots, not yours" (at the first time the player is hit, 4 s), "Health returns between waves" (at the first breather). They show on every run (there is no tutorial flag; the run is five minutes).

13. Scope

13.1 Cut list: deliberately OUT of 1.0

- **Reserved for the on camera iteration: one new enemy behaviour.** A second bandit type (for example a rusher on `Character_Hazmat` that runs straight in and punches, or a flanker that keeps out of the camera's view cone). What 1.0 prepares for it, and nothing more: the `WaveSpawner` reads the bandit prefab from an exported array `bandit_scenes` with one entry, and the wave table has a column for the prefab index that is 0 everywhere. No second scene, no second script, no second behaviour is written.
- **Reserved for the on camera iteration: one new weapon.** A second weapon prefab (for example a shotgun on the `Shotgun` mesh, or a grenade launcher on the `GrenadeLauncher` mesh with an arcing projectile). What 1.0 prepares for it, and nothing more: `Player/WeaponSlot` and the exported `weapon_scene` on the player. No swap key, no second prefab, no ammo system is written.
- Jump, crouch, sprint, dodge, melee, leaning.
- Ammo, reload, weapon heat, weapon pickups.

- Projectile weapons, explosions (`ExplodingBarrel`, `Grenade`, `FireGrenade`, `RocketLauncher` are decoration or unused), hazards (`Landmine`, `BearTrap_*`).
- A second arena, a boss, difficulty settings, endless mode.
- Controller support, iPad touch controls (Mac only in 1.0), rebinding.
- Head shots, damage numbers, kill streak multipliers.
- Leaderboards, Game Center, cloud save.
- Localisation beyond English.
- Any procedural placement of props.

13.2 Build order (each step ends with a run in Xogot, a screenshot check, and a commit)

1. Project settings check (already set: name, 1920 x 1080, `canvas_items`, `expand`, `fullscreen`, `MSAA`), main scene, the input map of section 5 through `xo inputmap`, autoloads `Game` and `Audio`, collision layer names. `tools/compare.py` written and run once against a black frame to prove the format. Run: an empty main scene shows black without errors.
2. `tools/gen_audio.py` and the WAV set, `--verify` passing. Commit the audio.
3. `main.tscn` with the fader; `arena.tscn` with `Environment`, `Sun`, `Ground`, `ConceptCam`; run from `ConceptCam:sky` and ground visible, comparison item 1 (horizon) checked by hand.
4. The concept region of 7.1 placed by hand, container materials, `ConceptDressing`; screenshot from `ConceptCam`; `comparison-01.png`; fix the first failing item of the check order, repeat until items 1 to 5 pass; `comparison-02.png`.
5. Lighting and environment values of 1.6, `tools/compare.py --values`; `comparison-03.png` with items 6 to 11 passing.
6. The rest of the yard, the perimeter, the navigation bake, `tests/test_arena.gd` passing; a top down editor screenshot of the whole yard saved to `docs/evidence/yard-top.png`; `comparison-04.png` unchanged from 03 inside the concept region.
7. `player.tscn` with the camera rig, movement, facing; `tests/test_camera.gd` and `tests/test_facing.gd` passing; the `snap_concept_view` action; run: the first frame of play matches `ConceptCam`.
8. `blaster.tscn`, tracer, flash, impact, recoil, `tests/test_weapon.gd`; run: shooting the tank leaves puffs.
9. `bandit.tscn` with the single behaviour, `wave_spawner.gd` with wave 1 only, damage both ways, health, vignette; `tests/test_bandit_bounds.gd` and `tests/test_cover.gd` passing; `comparison-05.png` from the concept view with the dressing.
10. All five waves, breathers, pickups, scoring, results screen, save file, pause.
11. `main_menu.tscn` with the orbit camera and the UI, the menu difference test; `comparison-06.png`.
12. Feedback and audio pass: every sound hooked up, music crossfades, ducking, announcements, tutorial messages, slow motion on the win; a full playthrough of the five waves watched through screenshots saved to `docs/evidence/run-wave-N.png`; tuning of counts and gaps; `comparison-final.png`.
13. README TBDs filled (controls, model, versions), final commit and push.

14. Missing assets and open questions

Need	Used by	Resolution
Audio (all of section 10)	everything	Synthesised WAVs from <code>tools/gen_audio.py</code> , committed under <code>res://assets/audio/</code> .
A blaster model	the player	The <code>AK</code> mesh inside <code>Character_Soldier</code> with its wood surface overridden to red (<code>res://materials/blaster_red.tres</code>).

Need	Used by	Resolution
Blue and yellow containers	concept region	Material overrides on the pack's containers (<code>res://materials/container_*.tres</code>), the files are not edited.
A tall water tower	concept region	<code>WaterTank_Platform</code> scaled 2.4 (8.4 units tall).
Rocks	concept region, right side	None in the pack: <code>Debris_Tires</code> at the rock's position.
Grass patches	ground	Flat green <code>PlaneMesh</code> patches at Y 0.02 (section 1.6).
Muzzle flash and impact textures	effects	A star drawn in code into an <code>ImageTexture</code> at start (<code>res://scripts/fx/textures.gd</code>), particles use built in <code>SphereMesh</code> .
Crosshair, health bar, vignette, edge marker	HUD	Drawn in code with <code>_draw()</code> and a small radial shader; no textures.
UI font	all screens	Godot's default font with outlines; a free font may be added later under <code>assets/fonts/</code> with its license file.
Bandit voice	death, hit	Synthesised tones, no voice samples.
Sky	environment	<code>ProceduralSkyMaterial</code> with the values of 1.6; no HDRI.
Chain link fence	perimeter	<code>Barrier_Large</code> (the hazard base panel of the concept). <code>MetalFence</code> is the plain panel and is not used.

Open questions for Marco (answered with the assumption in brackets): none that block the build. Assumptions: Mac only, keyboard and mouse; one save slot; the concept's bandits carry pistols (`Pistol` mesh); the working title is used on screen; the tank is decoration and cover, it never fires.

15. Definition of done and the master prompt

Definition of done for 1.0, in order:

- `docs/evidence/comparison-final.png` exists and passes the test of section 1.5 and the value table of 1.6.
- `xo test run` passes every suite in `tests/: test_scenes, test_arena, test_camera, test_facing, test_weapon, test_bandit_bounds, test_cover, test_menu`.
- `python3 tools/gen_audio.py --verify` passes.
- A full run from the menu through five waves to the results screen and back to the menu was played through `xo` and watched in screenshots (`docs/evidence/run-wave-1.png` to `run-wave-5.png` , `run-results.png`), and an overrun run reaches the results screen too.
- Every scene is a saved `.tscn` with the node names of section 8, every prop has collision, every character instance shows exactly one weapon mesh.
- `README.md` has its TBDs filled (controls, the building model, the version list).
- One commit per build order step, pushed.

Master prompt paragraph (what the building agent is told, in full): Build Container Yard from this document, in the Xogot editor through `xo` , following the build order of section 13.2 step by step, one commit per step, a run and a screenshot at the end of each. Section 1 is the acceptance test for the look: keep `docs/concept/concept.png` open, produce `docs/evidence/comparison-NN.png` after every visual milestone with `tools/compare.py` , judge it in the fixed check order and fix only the first failing item, until `comparison-final.png` passes. Every scene is a saved `.tscn` built in the editor with semantic node names; scripts add behaviour and never build the tree; bandits, pickups and effects are instanced from saved prefabs. Every 3D object is a real path from `docs/asset-inventory.md` ; the pack files are never

edited, colours change through material overrides. The camera has no position smoothing. Facing follows the camera yaw while aiming or firing, and the three facing tests pass. Bandits never leave the yard and never enter geometry, and the differential bounds test that fails a bandit which does not move passes. Audio is synthesised by `tools/gen_audio.py` and obeys the three rules: no chiptune or 8 bit anywhere; every music loop is seamless with at least 45 seconds of distinct material; every SFX is physical and satisfying with random pitch variation. The launch settings are 1920 x 1080, `canvas_items`, `expand`, fullscreen on Mac, and are in place before any UI is laid out. Version 1.0 has one arena, one player, one weapon, one enemy type with one behaviour, five waves, health, score, a menu with a moving camera through the built yard and a results screen; the second enemy behaviour and the second weapon are reserved for the on camera iteration and get only the slots described in 13.1. Do not add anything else.

16. Version 1.1: the full arsenal (supersedes 7.3, the weapon rows of 5, 8, 10, 11.2, 12, 13.1, 13.2 and 15)

Why. The Toon Shooter Game Kit ships fourteen weapon meshes inside every character file and fourteen matching loose models in `Guns/gltf/`: `AK`, `Pistol`, `Revolver`, `Revolver_Small`, `SMG`, `Shotgun`, `ShortCannon`, `Sniper`, `Sniper_2`, `GrenadeLauncher`, `RocketLauncher`, `Knife_1`, `Knife_2`, `Shovel`. Version 1.0 used one. Version 1.1 uses **all fourteen**: the player starts with the blaster and collects the other thirteen from weapon crates that appear between waves, so that a complete run puts every weapon of the kit in the player's hands. This is a firm requirement, not a nice to have: the definition of done (16.9) fails if any of the fourteen cannot be picked up and fired in one run. Bandits are unchanged (one type, one behaviour, pistol). The reserved weapon of 13.1 is withdrawn; the reserved enemy behaviour stays as the on camera iteration.

16.1 Inventory and switching (supersedes the weapon rows of section 5)

The player owns a list of weapons, in the fixed order of the table in 16.2. The blaster is owned from the start and never runs dry; every other weapon is owned once its crate has been picked up and carries its own ammo. Exactly one weapon is held; its mesh is the only visible weapon mesh on the character (the rule of section 9 still holds: one visible weapon mesh per character at all times).

Action name	Binding	Effect
<code>weapon_next</code> / <code>weapon_prev</code>	Mouse wheel down / up	Cycle through the owned weapons in table order, wrapping. Switching takes 0.25 s (<code>Idle_Shoot</code> blend, the old mesh hides and the new one shows at 0.12 s); firing during the switch is ignored.
<code>weapon_last</code>	Q	Swap to the previously held weapon.
<code>weapon_1</code> to <code>weapon_7</code>	1 to 7	Pick a category (16.2): 1 blaster, 2 sidearms, 3 automatic, 4 shotguns, 5 snipers, 6 launchers, 7 melee. Pressing the same key again cycles inside the category among the owned weapons. A category with nothing owned does nothing and the HUD flashes the empty slot.
<code>fire</code>	Left mouse button	As before. Semi automatic weapons fire once per press; automatic weapons fire while held; melee weapons swing once per press.
<code>aim</code>	Right mouse button	As before; for the snipers the aim FOV is the scope FOV of 16.2 instead of 40.

A weapon with zero ammo stays owned and selectable, plays the empty click on `fire` and shows "0" in red on the HUD; a new crate of the same weapon refills it to its full load. Ammo never drops from bandits.

16.2 The fourteen weapons (supersedes 7.3)

All hitscan weapons trace from the camera through the crosshair (section 7.3) with the listed cone; damage is per hit (per pellet for shotguns). Projectiles spawn at `Muzzle` aimed at the point the crosshair ray hits at 60 units (or the ray's end). Melee weapons hit every bandit inside the reach and arc in front of the player, once per swing, on the swing's contact frame (0.35 of the clip). A bandit has 30 health. Recoil kicks the camera pitch by the listed amount and returns over 0.12 s. All weapons use the tracer, flash and impact prefabs of 7.3 unless noted. Every weapon is its own prefab `res://scenes/weapons/<name>.tscn` (root `Node3D` named after the weapon) that only carries exported numbers and its sounds; the fourteen prefabs share one script `weapon.gd` with three modes (hitscan, projectile, melee). The mesh is the character's own mesh of the same name.

Cat.	Weapon (mesh)	Mode	Rate	Damage	Cone / reach	Ammo per crate	Recoil	Notes
1	Blaster (<code>AK</code> , red override)	hitscan, automatic	8 per s	10	1.2° (0.5° aimed)	infinite	0.6°	the starting weapon of 1.0, unchanged
2	Pistol (<code>Pistol</code>)	hitscan, semi	up to 4 per s	15	0.8°	36	0.8°	the bandits' gun
2	Revolver (<code>Revolver</code>)	hitscan, semi	up to 2.5 per s	35	0.4°	18	2.0°	one hit staggers: the bandit's <code>HitReact</code> lasts 0.6 s instead of 0.43
2	Small revolver (<code>Revolver_Small</code>)	hitscan, semi	up to 5 per s	18	1.0°	30	0.9°	
3	SMG (<code>SMG</code>)	hitscan, automatic	14 per s	6	2.5° (1.5° aimed)	120	0.3°	the cone grows by 0.1° per shot held, up to 4°, and resets 0.3 s after release
4	Shotgun (<code>Shotgun</code>)	hitscan, 8 pellets, semi	up to 1.2 per s	7 per pellet	7° cone, range 18	16 shells	3.0°	a pellet beyond 18 units does nothing
4	Sawn off (<code>ShortCannon</code>)	hitscan, 10 pellets, semi	up to 0.9 per s	8 per pellet	10° cone, range 12	12 shells	4.0°	knocks a bandit hit by 5 or more pellets back 1.0 unit
5	Sniper (<code>Sniper</code>)	hitscan, semi	up to 0.8 per s	90	0.1°, 0.05° aimed	10	2.5°	aim FOV 25 with a code drawn scope circle; hip fire cone 3°
5	Light sniper (<code>Sniper_2</code>)	hitscan, semi	up to 1.2 per s	60	0.2°, 0.1° aimed	14	1.8°	aim FOV 30; hip fire cone 2.5°
6	Grenade launcher (<code>GrenadeLauncher</code>)	projectile, arc	1 per s	60 at the centre falling to 20 at the edge, radius 3.0	speed 18, gravity 24	8	1.5°	the <code>Grenade</code> model as the projectile; bounces up to 2 times (restitution 0.4), explodes on touching a bandit or 1.5 s after launch
6	Rocket launcher (<code>RocketLauncher</code>)	projectile, straight	0.6 per s	120 at the centre falling to 30 at the edge, radius 4.0	speed 30, no gravity	4	3.0°	a <code>FireGrenade</code> model as the rocket with a smoke trail (<code>GPUParticles3D</code> , 30 per s, 0.6 s life); explodes on any contact

Cat.	Weapon (mesh)	Mode	Rate	Damage	Cone / reach	Ammo per crate	Recoil	Notes
7	Knife (Knife_1)	melee	2 swings per s	40	reach 1.8, arc 90°	infinite	0	Punch clip; the player runs 10 % faster while holding a melee weapon
7	Combat knife (Knife_2)	melee	2.5 swings per s	30	reach 1.6, arc 90°	infinite	0	as above
7	Shovel (Shovel)	melee	1.2 swings per s	70	reach 2.2, arc 120°	infinite	0	knocks the bandit back 1.5 units; Punch clip slowed to 0.7 speed

Explosions (launchers): `res://scenes/fx/explosion.tscn`: a one shot `GPUParticles3D` fireball (24 particles, `SphereMesh` 0.3, `#ffb347` to `#4a3a30` over 0.5 s), an `OmnLight3D` flash (range 8, energy 6, 0.08 s), a 0.25 s camera shake of 0.15 units, the explosion sound. Damage is applied to every bandit and to the player inside the radius, falling linearly from the centre value to the edge value; the player takes 50 % of it (self damage exists, so firing a rocket at the sandbag in front of you hurts). `ExplodingBarrel` still does not explode (cut list).

Ammo on the HUD (supersedes 11.2, bottom right): the held weapon's name (32 px) and its ammo as "12 / 16" (40 px) or "∞" for the blaster and the melee weapons; a row of seven category slots (36 x 36 px each, code drawn squares with the category number) showing owned categories filled and the held one highlighted. A weapon switch flashes the name for 0.3 s.

16.3 Weapon crates (new pickup)

`res://scenes/actors/weapon_crate.tscn`, root `WeaponCrate` (`Area3D`, layer 4, monitoring layer 2), a `Crate` model on the ground with the weapon's loose model from `Guns/gLTF/` (for example `res://assets/Toon Shooter Game Kit - Dec 2022/Guns/gLTF/Shotgun.glTF`) floating 0.6 units above it at scale 1.0, rotating 90° per second and bobbing ± 0.08 units over 1.6 s, an `OmnLight3D` `#ffd36b` energy 1.2 range 2.5, and a `Label3D` with the weapon's name 1.4 units above the crate, billboarded, 0.6 units of text height. An exported `weapon_id` selects the weapon. Walking into it gives the weapon with its full ammo (or refills it if owned), auto switches to it if the player is holding the blaster, plays the pickup sound and a gold burst, and frees the node. Crates persist until picked up. Crate points: eight `Marker3D` under `Arena/CratePoints`: `CrateA` (-6, 0, 10), `CrateB` (3, 0, 11), `CrateC` (11, 0, 6), `CrateD` (13, 0, -6), `CrateE` (-2, 0, -12), `CrateF` (-12, 0, -4), `CrateG` (-9, 0, 4), `CrateH` (8, 0, -13). Crate points and health pickup points never coincide.

16.4 The crate schedule (supersedes the wave table's implications in 12)

Crates appear at the start of each breather, at crate points that are empty, chosen with the wave's seed. Every weapon appears exactly once in a run, so a run that collects every crate holds all fourteen weapons before wave 5:

Breather after wave	Crates	Weapons
1	2	Pistol, Shotgun
2	3	SMG, Knife_1, Revolver
3	3	Sniper, GrenadeLauncher, Revolver_Small
4	5	RocketLauncher, Sniper_2, ShortCannon, Shovel, Knife_2

The wave table of section 12 stays as written for the first playthrough; the tuning pass after it may raise the bandit counts of waves 4 and 5 (up to 14 and 18) to pay for the heavier weapons, never their health or their damage. The

tutorial gains one message at the first breather: "Weapon crates: walk into them, switch with the wheel or 1 to 7".

16.5 Scenes (supersedes the weapon rows of section 8)

- `res://scenes/weapons/blaster.tscn` and the thirteen others (`pistol`, `revolver`, `revolver_small`, `smg`, `shotgun`, `short_cannon`, `sniper`, `sniper_2`, `grenade_launcher`, `rocket_launcher`, `knife_1`, `knife_2`, `shovel`), one prefab each, root Node3D named `Blaster`, `Pistol`, and so on, all with `res://scripts/weapons/weapon.gd` and their exported numbers set in the inspector (through `xo node property set`), with `Sounds` (`Fire`, `Empty`, `Switch`) as children.
- `res://scenes/weapons/grenade.tscn` (root `Grenade`, `RigidBody3D`, the `Grenade` model, sphere collision 0.2) and `res://scenes/weapons/rocket.tscn` (root `Rocket`, `Area3D` moved by script, the `FireGrenade` model, the smoke trail).
- `res://scenes/fx/explosion.tscn`.
- `res://scenes/actors/weapon_crate.tscn`.
- `Player/WeaponSlot` holds all fourteen weapon prefab instances as children, named after the weapon; only the held one processes. `Player` exports nothing about weapons any more; the inventory is the list of owned weapon names, saved nowhere (a run starts with the blaster).
- `Arena/CratePoints` with the eight markers; `Arena/Pickups` is the runtime parent of crates and health pickups.

Signals: `Weapon.fired(kind, origin, hit_point, hit_body)` for hitscan and melee, `Weapon.launched(projectile)` for projectiles, `Projectile.exploded(position, radius, damage_centre, damage_edge)` → `Arena.on_explosion`, `WeaponCrate.collected(weapon_id)` → `Player.give_weapon`.

16.6 Audio (adds to section 10; the three hard rules apply unchanged)

Each weapon gets its own fire sound from `tools/gen_audio.py`, physical, with a transient, a body and a tail, pitch varied $\pm 6\%$ per shot: the pistol crack (60 ms), the revolver's heavier bang with a 250 ms tail, the small revolver's snappy pop, the SMG's short dry rattle per shot, the shotgun's boom with a 300 ms low tail, the sawn off's wider boom, the sniper's long crack with a 500 ms echo, the light sniper's shorter one, the grenade launcher's hollow thunk, the rocket's whoosh and ignition roar, the knives' swish (two variants) and a wet hit, the shovel's whoosh and a metallic clang on a hit. Plus: the explosion (a 700 ms low boom with debris crackle), the grenade bounce (a metallic tick), the empty click, the weapon switch (a mechanical clack), the crate pickup (a rising three note chime, distinct from the health chime). `--verify` checks every new file under the same rules.

16.7 Build order (adds to 13.2)

Step 8 becomes: the blaster and the shared `weapon.gd`, the tracer, the flash, the impact and the recoil, then the thirteen other prefabs in table order with their numbers, the projectiles and the explosion, the weapon crate, the HUD ammo block and the category slots, the switching, the crate schedule; `tests/test_weapons.gd` passing; run: pick up a shotgun crate placed by hand, fire it at the tank, see ten puffs; fire a rocket at the sandbag line, see the explosion and take self damage. Everything else in 13.2 keeps its number.

16.8 Tests

- `tests/test_weapons.gd`: for each of the fourteen weapons, from a fresh arena with a bandit 12 units in front of the camera in the open: give the weapon with full ammo, fire for 1.0 s (or one swing for melee, one shot for the launchers), and assert the bandit's damage taken is within 20 % of the table's expected value for that second (rate x damage x hit fraction, hit fraction 1.0 for cones under 1.5° at 12 units, 0.8 for the SMG, per pellet 0.7 for

- the shotguns, the launchers' centre damage, melee with the bandit at 1.2 units); assert the ammo counter decreased by the right amount; assert exactly one weapon mesh is visible on the player after the switch.
- `tests/test_crates.gd`: the crate schedule of 16.4 yields every one of the thirteen weapons exactly once over the four breathers, at empty crate points only; a scripted player that walks into every crate as it appears owns fourteen weapons before wave 5 starts.
 - `tests/test_switch.gd`: the wheel cycles through the owned weapons in table order and wraps; category keys cycle inside their category; Q swaps back; no input during the 0.25 s switch fires a shot.
 - Pillar 2's zero lag test and the three facing tests run with every weapon held (the facing rule of 7.2 does not depend on the weapon).

16.9 Definition of done and the master prompt (adds to 15)

Item 2 of the definition of done adds `test_weapons`, `test_crates` and `test_switch`. A new item 8: a run watched through screenshots in which the player picks up every crate and fires every one of the fourteen weapons at least once (`docs/evidence/weapon-<name>.png`, fourteen files). The master prompt of section 15 changes one sentence: "Version 1.1 has one arena, one player, **all fourteen weapons of the kit collected from crates between waves as in section 16**, one enemy type with one behaviour, five waves, health, score, a menu with a moving camera through the built yard and a results screen; the second enemy behaviour is reserved for the on camera iteration and gets only the spawner slot described in 13.1. Do not add anything else."